

Caption: Smart Lender

Project Description:

The **Smart Lender** project is a machine learning–based web application developed using Python and the Flask framework to predict loan eligibility. It analyzes applicant details such as income, loan amount, credit history, employment status, and other relevant factors to determine whether a loan is likely to be approved or rejected. The application provides users with a simple and interactive interface for entering their information and viewing the prediction result.

The project follows a modular architecture with separate components for data preprocessing, model training, and web application development. Multiple machine learning algorithms are trained and compared, and the best-performing model is selected for prediction. The application is designed to be easy to use, maintain, and extend, making it a reliable solution for demonstrating machine learning–based loan eligibility prediction.

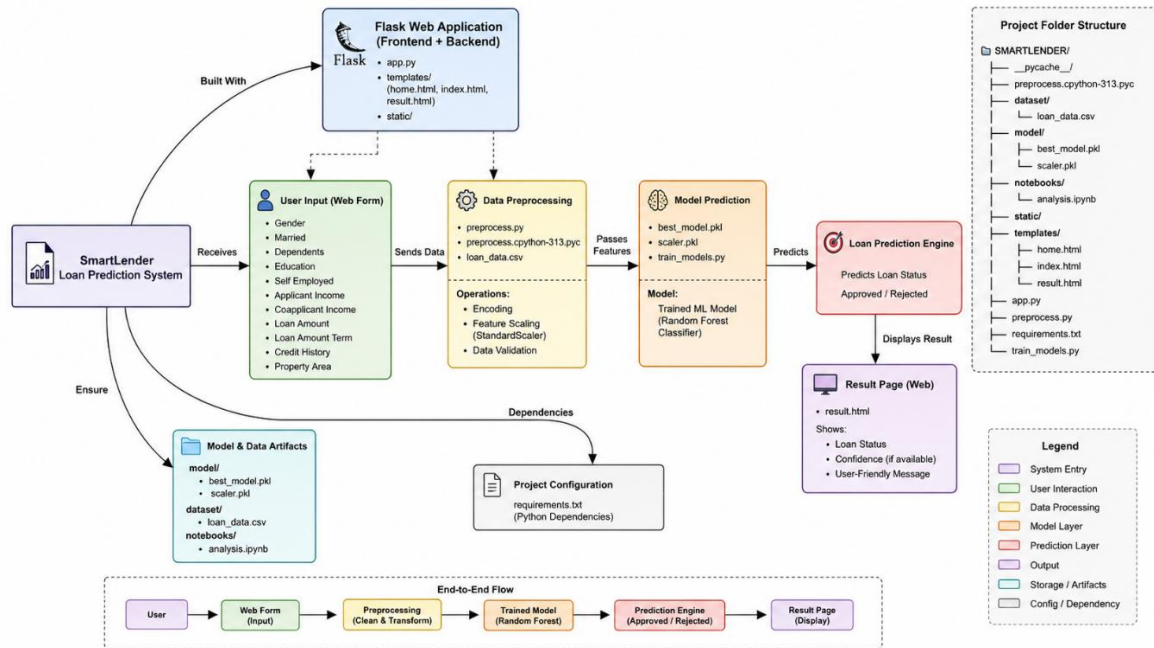
Scenarios

Scenario 1: A user with a stable income, good credit history, and a moderate loan requirement enters their details into the Smart Lender application. The system processes the information using the trained machine learning model and predicts that the loan is **Approved**, displaying the result along with the confidence score.

Scenario 2: A user with a low income, poor credit history, and a high loan amount submits their application through the Smart Lender system. The application analyzes the entered details and predicts that the loan is **Rejected**, helping the user understand their loan eligibility before applying to a bank.

Scenario 3: A user enters applicant details such as gender, marital status, dependents, income, loan amount, loan term, credit history, and property area into the Smart Lender application. The system validates the inputs, preprocesses the data, and instantly provides a loan eligibility prediction, enabling a quick and reliable decision-making process.

Technical Architecture:



The technical architecture illustrates the workflow of the Smart Lender Loan Prediction System. Users enter loan application details through a web form developed using Flask. The input data is sent to the data preprocessing module, where it is validated, encoded, and scaled before being passed to the trained Random Forest machine learning model (best_model.pkl). The model predicts whether the loan should be approved or rejected, and the result is displayed on the result page. The architecture also highlights the project structure, including the dataset, model files, templates, static resources, notebooks, and configuration files, demonstrating the complete end-to-end flow of the application.

Pre-requisites:

The project was developed using a set of powerful Python-based tools, libraries, and web technologies for data preprocessing, machine learning, model deployment, and web application development. These technologies provide the foundation for efficient model training, prediction, and user-friendly deployment.

Visual Studio Code: A lightweight and powerful source code editor used for developing the Flask application, machine learning scripts, and frontend components.

<https://code.visualstudio.com/>

Python: A high-level programming language used for data preprocessing, machine learning model development, and backend application logic.

<https://www.python.org/>

NumPy: A fundamental library for numerical computing in Python, providing support for multidimensional arrays and mathematical operations.

<https://numpy.org/doc/stable/>

Pandas: A powerful data analysis and manipulation library used for loading, cleaning, and preprocessing the loan dataset.

<https://pandas.pydata.org/docs/>

Scikit-learn: A machine learning library used for data preprocessing, feature scaling, training the Random Forest classifier, and evaluating model performance.

<https://scikit-learn.org/stable/>

Joblib: A Python library used for saving and loading trained machine learning models and preprocessing objects efficiently.

<https://joblib.readthedocs.io/>

Flask: A lightweight Python web framework used to build the SmartLender web application and integrate the trained machine learning model for loan prediction.

<https://flask.palletsprojects.com/>

HTML, CSS, and JavaScript: Web technologies used to design responsive, interactive, and user-friendly interfaces for the loan application and prediction result pages.

<https://www.w3schools.com/>

Jupyter Notebook: An interactive development environment used for exploratory data analysis, model training, experimentation, and performance evaluation.

<https://jupyter.org/>

Git: A distributed version control system used to track source code changes, manage project versions, and support collaborative development.

<https://git-scm.com/doc>

The project is developed through a structured workflow consisting of multiple epics, covering data collection, analysis, preprocessing, model development, and deployment. Each epic focuses on a specific stage of the machine learning lifecycle to ensure systematic and efficient project execution.

Project Workflow:

Epic 1: Data Collection and Architecture Design

Activity 1.1: Download the required dataset from the provided source and store it in the project directory.

Activity 1.2: Define the application architecture, including data flow, machine learning workflow, and deployment structure.

Epic 2: Visualizing and Analyzing the Data

Activity 2.1: Import and read the dataset using Pandas for further analysis.

Activity 2.2: Perform univariate analysis to understand the distribution of individual features.

Activity 2.3: Conduct bivariate analysis to identify relationships between two variables.

Activity 2.4: Perform multivariate analysis to uncover patterns involving multiple features.

Epic 3: Data Pre-Processing

Activity 3.1: Check for missing values, duplicates, and inconsistencies, and handle them appropriately.

Activity 3.2: Balance the dataset to ensure fair representation of all target classes.

Activity 3.3: Scale and normalize numerical features to improve model performance.

Activity 3.4: Split the dataset into training and testing sets for model development and evaluation.

Epic 4: Model Building

Activity 4.1: Train and evaluate different Machine Learning Models on the prepared dataset.

Activity 4.2: Evaluate all models using appropriate metrics and save the best-performing model for deployment.

Epic 5: Application Building

Activity 5.1: Design and develop HTML pages to create a user-friendly interface.

Activity 5.2: Build the Flask application and integrate the trained machine learning model.

Activity 5.3: Run, test, and validate the application to ensure accurate predictions and smooth functionality.

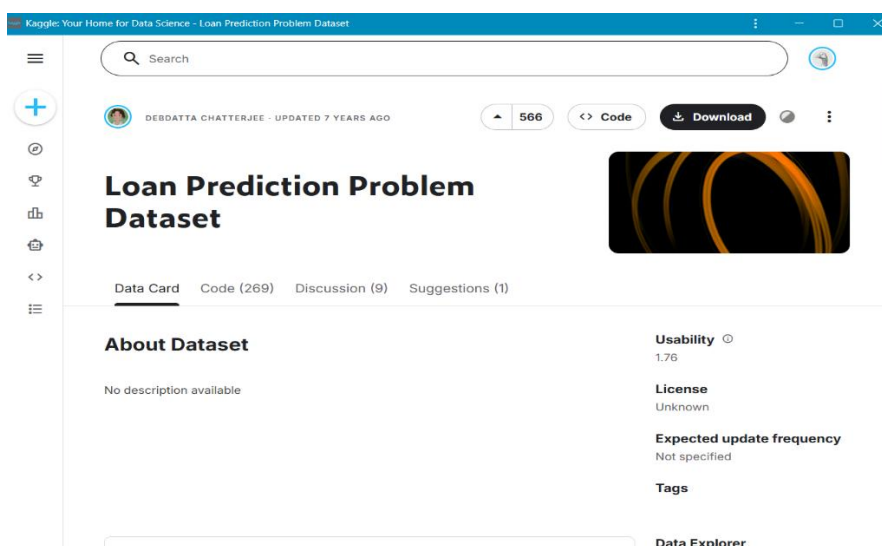
Epic 1: Data Collection and Architecture Design

The project began by collecting the **Loan Approval Prediction** dataset containing applicant and loan details. The dataset was analyzed to identify the input features and the target variable (**Loan_Status**). A technical architecture was designed to define the flow from user input to loan prediction. The project structure was organized into templates, static files, datasets, and machine learning models. This phase established the foundation for building and deploying the Smart Lender application.

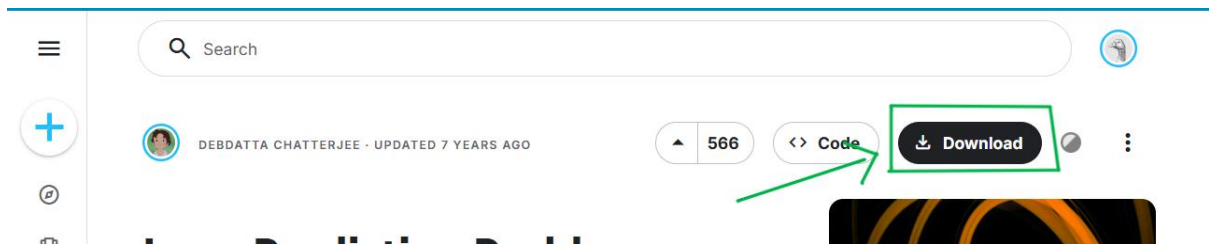
Activity 1: Download the Dataset

The dataset used for the SmartLender – Loan Prediction using Machine Learning project was downloaded from the Kaggle platform. It contains historical loan application records with applicant details such as gender, marital status, education, applicant income, co-applicant income, loan amount, credit history, property area, and loan status. This dataset was used to train and evaluate the machine learning model for predicting loan approval.

Step-1: Open the Kaggle and search for the Loan Prediction Dataset and open it.



Step-2: Once you have opened the Loan Prediction data set click on download button.

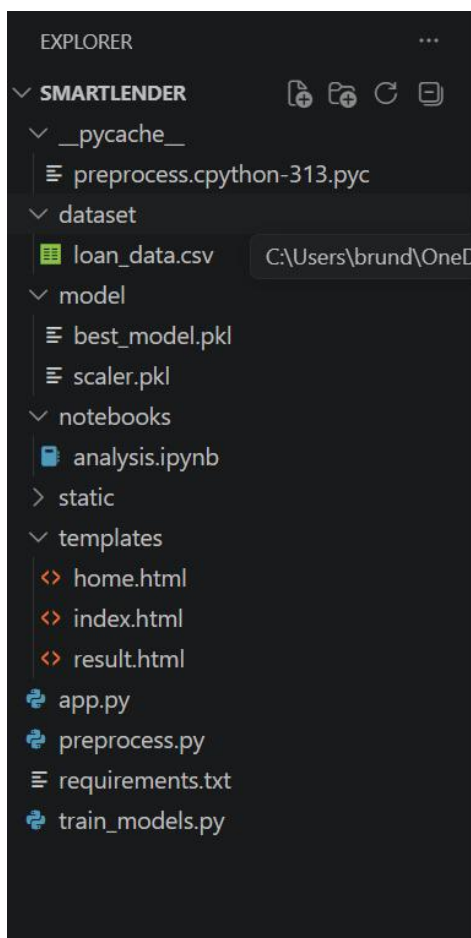


Step-3: The data set will be downloaded in the Zip Format. Unzip it extract all into a specific folder in that you will be able to see Training dataset and the testing dataset.

Step-4: copy the Training Dataset and paste it in the platform where you are going to develop the code.

Activity 2: Defining the Application Architecture

After data collection, a modular and scalable application architecture was designed to ensure separation of concerns, maintainability, and future extensibility. The system follows a three-tier architecture, consisting of the user interface, backend API layer, and AI/ML service modules. A well-structured directory hierarchy was adopted to separate frontend, backend, model, and testing components, improving code organization, development efficiency, and scalability.



This is the Architecture of the project as the project contains many comparisons and graphs related so, many sub folders got created in this. The Analysis file contains the comparison graphs of all the features .and the HTML pages will add the bit clarity to the project result. And the style.css will be produced in the static folder, With this we can add more professionalism for the project.

Epic 2: Visualizing and Analyzing the Data:

Activity 2.1: Import and Read Dataset:

The loan prediction dataset was imported into the project using the **Pandas** library in Python. The dataset (loan_data.csv) was loaded into a DataFrame using the read_csv () function for further analysis and preprocessing. This step allows the application to access applicant information, inspect the data structure, and prepare it for machine learning model training. Reading the dataset is the first step in building an accurate loan prediction system.

1. Import all the datatypes required for the dataset.

```
> import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

# Load dataset
df = pd.read_csv('../dataset/loan_data.csv')

print("Shape:", df.shape)
print("\nFirst 5 rows:")
print(df.head())

print("\nData types:")
print(df.dtypes)

print("\nBasic statistics:")
print(df.describe())

print("\nMissing values:")
print(df.isnull().sum())
```

[1]

2. When we run the above cell we will be able to see the dataset that we have imported into the project.

```

Shape: (614, 13)

First 5 rows:
  Loan_ID Gender Married Dependents Education Self_Employed \
0 LP001002 Male No 0 Graduate No
1 LP001003 Male Yes 1 Graduate No
2 LP001005 Male Yes 0 Graduate Yes
3 LP001006 Male Yes 0 Not Graduate No
4 LP001008 Male No 0 Graduate No

  ApplicantIncome CoapplicantIncome LoanAmount Loan_Amount_Term \
0 5849 0.0 NaN 360.0
1 4583 1508.0 128.0 360.0
2 3000 0.0 66.0 360.0
3 2583 2358.0 120.0 360.0
4 6000 0.0 141.0 360.0

  Credit_History Property_Area Loan_Status
0 1.0 Urban Y
1 1.0 Rural N
2 1.0 Urban Y
3 1.0 Urban Y
4 1.0 Urban Y

Data types:
...
Credit_History 50
Property_Area 0
Loan_Status 0
dtype: int64

```

Activity 2.2: Univariate Analysis

Univariate analysis was performed to examine each feature in the loan dataset individually. It helps in understanding the distribution, frequency, and characteristics of variables such as applicant income, loan amount, education, credit history, and loan status. Graphs like histograms and count plots were used to identify patterns, detect missing values, and understand the overall data distribution before preprocessing and model training.

```

#Categorical
cat_cols = ['Gender', 'Married', 'Education', 'Self_Employed',
            'Credit_History', 'Property_Area', 'Loan_Status']

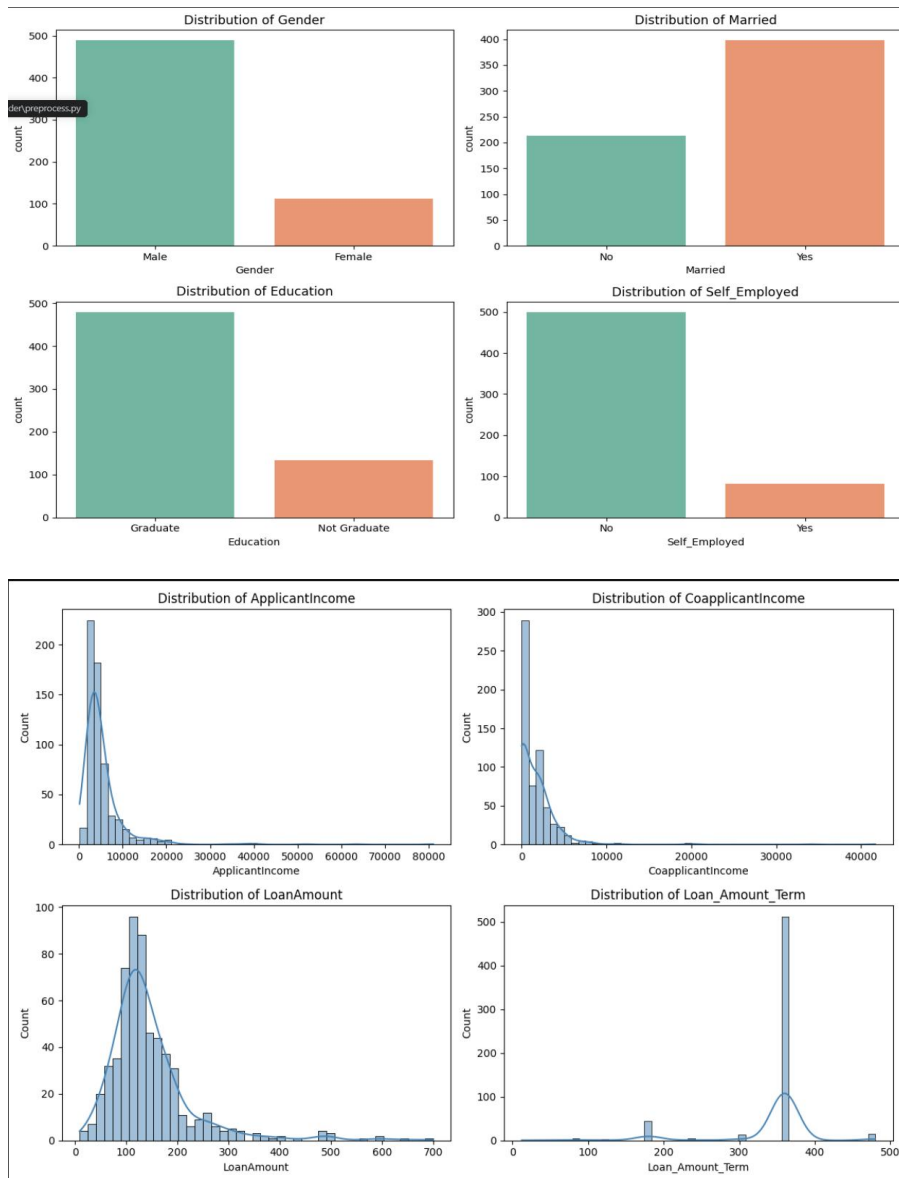
for i in range(0, len(cat_cols), 4):
    batch = cat_cols[i:i+4]
    fig, axes = plt.subplots(2, 2, figsize=(12, 8))
    axes = axes.flatten()
    for j, col in enumerate(batch):
        sns.countplot(x=col, data=df, ax=axes[j], palette='Set2')
        axes[j].set_title(f'Distribution of {col}')
    for k in range(j+1, 4):
        axes[k].set_visible(False)
    plt.tight_layout()
    plt.show()

#Numerical
num_cols = ['ApplicantIncome', 'CoapplicantIncome', 'LoanAmount', 'Loan_Amount_Term']

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
axes = axes.flatten()
for i, col in enumerate(num_cols):
    sns.histplot(df[col].dropna(), kde=True, ax=axes[i], color='steelblue')
    axes[i].set_title(f'Distribution of {col}')
plt.tight_layout()
plt.show()

```

After writing the code when we run the code we get the comparison graphs of each and every feature in the graph format.



These are the Comparison graphs of different features based on the Univariate Analysis.

Activity 2.3: Bivariate Analysis

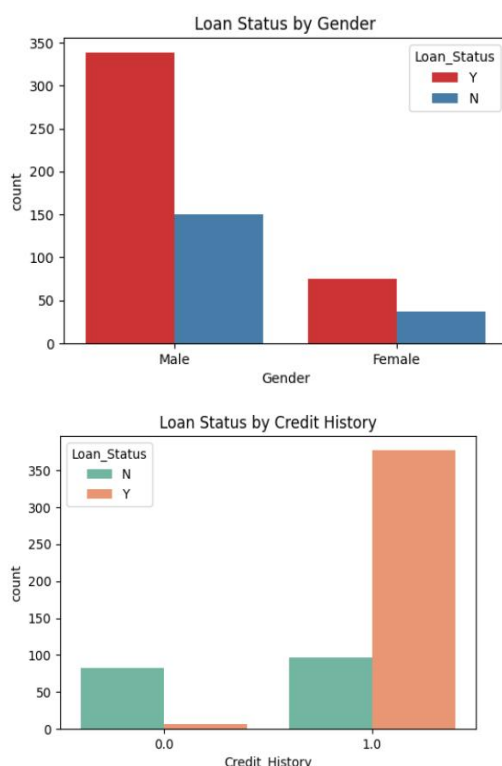
Bivariate analysis was performed to study the relationship between two variables in the loan dataset. It helps identify how features such as **credit history**, **applicant income**, **education**, and **property area** influence the **loan approval status**. Visualizations like bar charts, box plots, and scatter plots were used to analyze these relationships and gain insights that support accurate model

development.

```
plt.figure(figsize=(6, 4))
sns.countplot(x='Gender', hue='Loan_Status', data=df, palette='Set1')
plt.title('Loan Status by Gender')
plt.savefig('../static/bivariate_gender.png')
plt.show()

# Loan Status vs Credit History
plt.figure(figsize=(6, 4))
sns.countplot(x='Credit_History', hue='Loan_Status', data=df, palette='Set2')
plt.title('Loan Status by Credit History')
plt.savefig('../static/bivariate_credit.png')
plt.show()
```

Once we run the above block of the code, we get the comparison graph with two features comparing their present features.

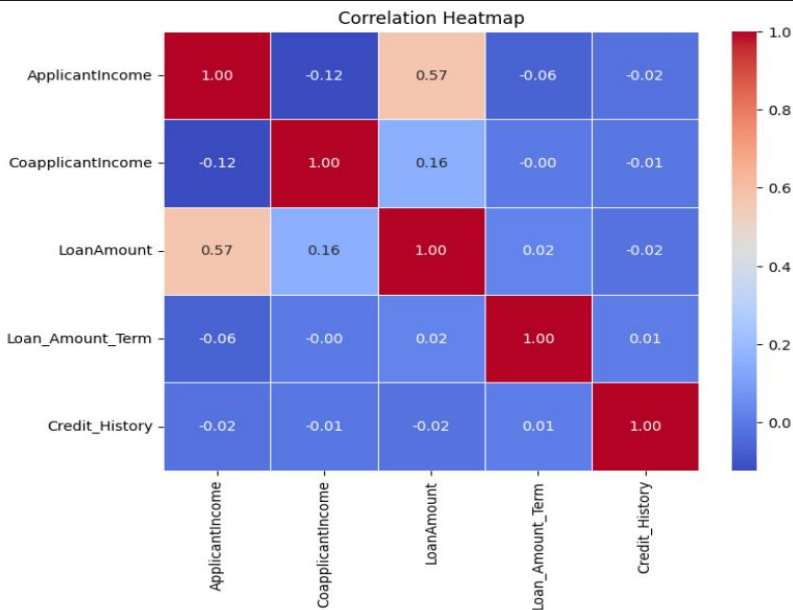


Activity 2.4: Multivariate Analysis

Multivariate analysis was performed to examine the relationship among multiple features in the loan dataset simultaneously. It helps identify how factors such as **applicant income, loan amount, credit history, education, and property area** together influence the **loan approval status**. Correlation analysis and visualization techniques were used to understand feature interactions and improve the performance of the machine learning model.

```
# Correlation heatmap (numerical features)
plt.figure(figsize=(8, 6))
numeric_df = df[['ApplicantIncome', 'CoapplicantIncome', 'LoanAmount',
                  'Loan_Amount_Term', 'Credit_History']].dropna()
corr = numeric_df.corr()
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', linewidths=0.5)
plt.title('Correlation Heatmap')
plt.savefig('../static/multivariate_heatmap.png')
plt.show()

# Pairplot
sns.pairplot(df[['ApplicantIncome', 'LoanAmount', 'CoapplicantIncome', 'Loan_Status']],
             hue='Loan_Status', palette='Set1', diag_kind='kde')
plt.savefig('../static/multivariate_pairplot.png')
plt.show()
```



Epic 3: Data Pre-Processing:

Activity 3.1: Checking and Handling Values

Handling Categorical Values

To prepare the dataset for machine learning, categorical features were converted into numerical values using encoding techniques. Columns such as Gender, Married, Dependents, Self_Employed, Credit_History, and other relevant attributes were encoded to ensure compatibility with machine learning algorithms.

The dataset was then inspected using methods such as `df.shape`, and `df.is null().sum()` to identify missing values and understand data types.

Missing Value Treatment:

Missing values in numerical features were replaced with the mean of the respective column. Missing values in categorical features were replaced with the mode (most frequent value).

This preprocessing step improved data quality and ensured the dataset was suitable for accurate model training and prediction.

```
# — Handle missing values —
# Categorical: fill with mode
cat_cols = ['Gender', 'Married', 'Dependents', 'Self_Employed', 'Credit_History']
for col in cat_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)

# Numerical: fill with mean
num_cols = ['LoanAmount', 'Loan_Amount_Term']
for col in num_cols:
    df[col].fillna(df[col].mean(), inplace=True)

print("Missing values after handling:\n", df.isnull().sum())

# — Encode categorical features —
le = LabelEncoder()
encode_cols = ['Gender', 'Married', 'Dependents', 'Education',
               'Self_Employed', 'Property_Area', 'Loan_Status']
for col in encode_cols:
    df[col] = le.fit_transform(df[col].astype(str))

# — Features and target —
X = df.drop(columns=['Loan_Status'])
y = df['Loan_Status']

print("\nClass distribution before SMOTE:", y.value_counts().to_dict())
```

Activity 3.2: Balancing the Dataset

Balancing the dataset is an important preprocessing step that ensures the machine learning model learns equally from both approved and rejected loan applications. The distribution of the Loan_Status classes was analyzed to check whether one class contained significantly more records than the other. An imbalanced dataset can cause the model to favor the majority class, leading to biased predictions.

To improve the model's performance, appropriate balancing techniques were considered whenever necessary. A balanced dataset helps the model accurately identify both loan approvals and rejections, resulting in better prediction accuracy, reduced bias, and improved reliability of the SmartLender loan prediction system.

```
# — Balance dataset with SMOTE —
smote = SMOTE(random_state=42)
X_res, y_res = smote.fit_resample(X, y)
print("Class distribution after SMOTE:", pd.Series(y_res).value_counts().to_dict())
```

In this we have used SMOTE Key word that generally helps in balancing the dataset. It will automatically add the missing data values so that the dataset becomes balanced.

Activity3.3: Scaling the Data:

After completing data preprocessing and balancing, the next step was feature scaling. The dataset contains numerical features such as Applicant Income, Co-applicant Income, Loan Amount, and Loan Amount Term, which have different value ranges. If these features are used directly, variables with larger values may have a greater influence on the machine learning model than those with smaller values. Therefore, feature scaling is performed to bring all numerical features to a common scale.

In this project, the StandardScaler class from the Scikit-learn library was used to standardize the input features. StandardScaler transforms the data by subtracting the mean and dividing by the standard deviation, resulting in features with a mean of 0 and a standard deviation of 1. This preprocessing step improves the efficiency of model training and ensures consistent feature representation.

```
# — Scale features —  
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X_res)  
  
# Save scaler for use in Flask app  
joblib.dump(scaler, 'model/scaler.pkl')
```

The scaled dataset was stored in the variable **X_scaled**, which was then used as the input for training the machine learning model. To maintain consistency during prediction, the fitted **StandardScaler** object was saved as **scaler.pkl** using the **Joblib** library. This saved scaler is later loaded in the Flask application to transform new user input data using the same scaling parameters that were applied during training.

Using the same scaler for both training and prediction ensures that the model receives standardized input, resulting in more accurate, reliable, and consistent loan approval predictions. This step plays an important role in improving the overall performance of the **SmartLender Loan Prediction System**.

Activity 3.4: Splitting Data into Training and Test Sets

Once the data was preprocessed and scaled, it was divided into training and testing datasets. This step is essential to evaluate the performance of the machine learning model on unseen data. The `train_test_split()` function from the Scikit-learn library was used to split the dataset into two parts. In this project, 80% of the data was allocated for training the model, while the remaining 20% was reserved for testing its performance.

A random state of 42 was specified to ensure that the data is split consistently every time the code is executed. This makes the results reproducible and helps in comparing the performance of different machine learning models under the same conditions.

```
# — Train/test split —  
X_train, X_test, y_train, y_test = train_test_split(  
    X_scaled, y_res, test_size=0.2, random_state=42  
)  
  
print(f"\nTrain size: {X_train.shape}, Test size: {X_test.shape}")  
return X_train, X_test, y_train, y_test, scaler
```

The training dataset (**X_train** and **y_train**) was used to train the Random Forest machine learning model, enabling it to learn patterns from the loan application data. The testing dataset (**X_test** and **y_test**) was kept separate and used only to evaluate the model's prediction performance on unseen data.

Finally, the sizes of the training and testing datasets were displayed to verify that the data was split correctly. This step ensures that the SmartLender Loan Prediction System is trained effectively and evaluated fairly, resulting in a reliable and accurate loan approval prediction model.

Epic 4: Model Building

Activity 4.1: Training different Machine Learning Models

To identify the most accurate model for loan prediction, multiple machine learning algorithms were trained and evaluated using the preprocessed dataset. Each model was trained using the training dataset and tested on the testing dataset. Their performance was compared based on training accuracy, testing accuracy, classification report, and confusion matrix. The model with the highest testing accuracy was selected as the final model for deployment in the SmartLender application.

1. Decision Tree Classifier

The **Decision Tree Classifier** is a supervised machine learning algorithm that predicts loan approval by creating a tree-like structure of decision rules. It divides the dataset into smaller subsets based on the most significant features, making the prediction process simple and easy to interpret. The model was trained on the training dataset and evaluated using the testing dataset to measure its performance.

2. Random Forest Classifier

The **Random Forest Classifier** is an ensemble learning algorithm that combines multiple decision trees to produce more accurate and reliable predictions. Each tree is trained on a different subset of the data, and the final prediction is determined through majority voting. This approach reduces overfitting and improves the overall performance of the model. Based on the evaluation results, the Random Forest model achieved the highest testing accuracy and was selected as the final model for the SmartLender Loan Prediction System.

```
def main():
    X_train, X_test, y_train, y_test, _ = preprocess()

    models = {
        'Decision Tree': DecisionTreeClassifier(random_state=42),
        'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
        'KNN': KNeighborsClassifier(n_neighbors=5),
        'XGBoost': XGBClassifier(n_estimators=100, learning_rate=0.1,
                                use_label_encoder=False,
                                eval_metric='logloss', random_state=42),
    }

    results = {}
    best_name, best_acc, best_model = '', 0, None

    for name, model in models.items():
        train_acc, test_acc, trained = evaluate_model(
            name, model, X_train, X_test, y_train, y_test
        )
        results[name] = {'train': train_acc, 'test': test_acc}
        if test_acc > best_acc:
            best_acc = test_acc
            best_name = name
            best_model = trained
```

3. K-Nearest Neighbors (KNN)

The **K-Nearest Neighbors (KNN)** algorithm classifies loan applications by comparing them with the nearest data points in the training dataset. It identifies the closest neighbors based

on feature similarity and predicts the loan status using majority voting among those neighbors. The value of **K = 5** was used in this project to achieve a balance between accuracy and prediction stability.

4. XGBoost Classifier

The **Extreme Gradient Boosting (XGBoost)** algorithm is an advanced ensemble learning technique that builds multiple decision trees sequentially. Each new tree attempts to correct the errors made by the previous trees, resulting in improved prediction accuracy. XGBoost is known for its efficiency, speed, and ability to handle complex datasets, making it one of the strongest models evaluated in this project.

Activity 4.2: Evaluating Performance and Saving the Model:

After training the machine learning models, their performance was evaluated to determine the most suitable model for loan prediction. Each model was tested using the testing dataset, and its performance was measured based on **training accuracy**, **testing accuracy**, **classification report**, and **confusion matrix**. These evaluation metrics help analyze the model's ability to correctly classify loan applications as **Approved** or **Rejected**.

To compare the performance of all models, a bar chart was generated showing the training and testing accuracy of each algorithm. This comparison made it easier to identify the model with the highest prediction accuracy and the best generalization performance.

```
print(f"\n✅ Best model: {best_name} (Test Accuracy: {best_acc*100:.2f}%)")
joblib.dump(best_model, 'model/best_model.pkl')
print("✅ Model saved to model/best_model.pkl")
```

Based on the evaluation results, the **Random Forest Classifier** achieved the highest testing accuracy among all the trained models and was selected as the final model for the SmartLender Loan Prediction System. The selected model was then saved as **best_model.pkl** using the **Joblib** library. Saving the trained model eliminates the need for retraining each time the application runs and enables the Flask web application to load the model directly for real-time loan approval predictions.

The comparison chart and confusion matrices generated for each model provide a clear visualization of their performance, making it easier to validate the effectiveness of the selected model. This evaluation process ensures that the deployed model is accurate, reliable, and capable of making consistent loan approval predictions for new user inputs.

Comparison Graph:

After evaluating all the machine learning models, a comparison graph was generated to visualize their performance. The graph compares the **training accuracy** and **testing accuracy** of the **Decision Tree**, **Random Forest**, **K-Nearest Neighbors (KNN)**, and **XGBoost** models. This visual comparison helps in identifying the model that provides the best prediction performance while maintaining good generalization on unseen data.

```
# — Comparison chart —
names = list(results.keys())
train_scores = [results[n]['train']*100 for n in names]
test_scores = [results[n]['test']*100 for n in names]

x = np.arange(len(names))
fig, ax = plt.subplots(figsize=(10, 5))
bars1 = ax.bar(x - 0.2, train_scores, 0.4, label='Train', color='steelblue')
bars2 = ax.bar(x + 0.2, test_scores, 0.4, label='Test', color='coral')
ax.set_xticks(x)
ax.set_xticklabels(names)
ax.set_ylabel('Accuracy (%)')
ax.set_title('Model Comparison – Train vs Test Accuracy')
ax.legend()
ax.bar_label(bars1, fmt='%.1f%%', padding=3)
ax.bar_label(bars2, fmt='%.1f%%', padding=3)
ax.set_ylim(0, 110)
plt.tight_layout()
plt.savefig('static/model_comparison.png')
plt.show()
```

The comparison graph clearly shows the performance of each machine learning model on both the training and testing datasets. Among all the models, the **Random Forest Classifier** achieved the highest testing accuracy while maintaining excellent training accuracy, making it the most reliable model for predicting loan approval. Therefore, the Random Forest model was selected as the final model and saved as **best_model.pkl** for deployment in the SmartLender Loan Prediction System.

```
PS C:\Users\brund\OneDrive\Desktop\SmartLender> & "C:\Program Files\Python313\python.exe" c:/Users/brund/OneDrive/Desktop/SmartLender/train_models.py
weighted avg    0.81    0.80    0.80    169

=====
Model: KNN
Train Accuracy: 82.96%
Test Accuracy: 69.23%
      precision    recall  f1-score   support

   Rejected      0.78      0.63      0.69      94
   Approved      0.62      0.77      0.69      75

 accuracy      0.69      169
 macro avg      0.70      0.69      0.69      169
weighted avg      0.71      0.69      0.69      169

C:\Users\brund\AppData\Roaming\Python\Python313\site-packages\xgboost\tuning.py:200: UserWarning: [11:43:03] WARNING: C:\actions-runner\_work\xgboost\xgboost\src\learner.cc:793:
Parameters: { "use_label_encoder" } are not used.

    bst.update(dtrain, iteration=i, fobj=obj)

=====
Model: XGBoost
Train Accuracy: 98.37%
Test Accuracy: 78.11%
      precision    recall  f1-score   support

   Rejected      0.88      0.70      0.78      94
   Approved      0.70      0.88      0.78      75

 accuracy      0.78      169
 macro avg      0.79      0.79      0.78      169
weighted avg      0.80      0.78      0.78      169

[+] Best model: Random Forest (Test Accuracy: 79.88%)
[+] Model saved to model/best_model.pkl
```



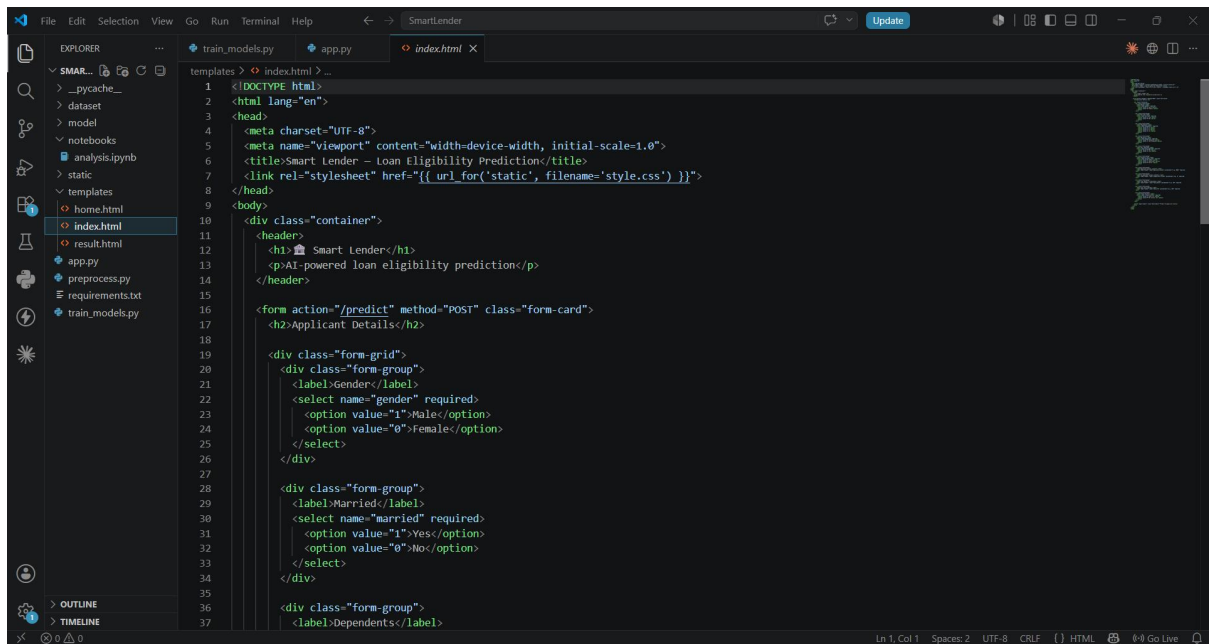
The graph also demonstrates that comparing multiple algorithms is an important step in selecting the most suitable model. It provides a clear visual representation of each model's performance, enabling the selection of the most accurate and robust algorithm for real-time loan prediction.

Epic 5: Application Building

Activity 5.1: Building HTML Pages

The user interface of the SmartLender Loan Prediction System was developed using HTML and CSS to provide a simple, responsive, and user-friendly experience. Multiple web pages were created to guide users through the loan prediction process. These pages include the Home Page, Loan Application Form, and Result Page. Each page was designed to ensure easy navigation and a professional appearance.

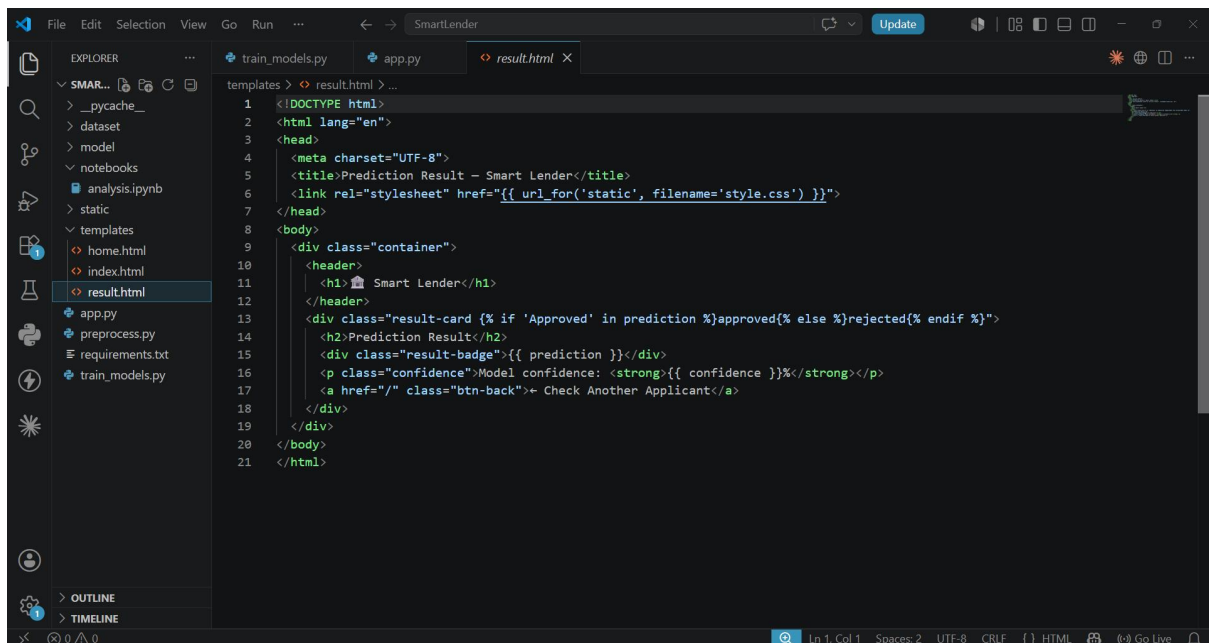
The HTML pages were integrated with the Flask backend, allowing users to enter loan details, submit the application, and receive instant loan approval predictions. A clean layout, input validation, and responsive design were implemented to improve usability and accessibility.



```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Smart Lender - Loan Eligibility Prediction</title>
7   <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
8 </head>
9 <body>
10  <div class="container">
11    <header>
12      <h1> Smart Lender</h1>
13      <p>AI-powered loan eligibility prediction</p>
14    </header>
15
16    <form action="/predict" method="POST" class="form-card">
17      <h2>Applicant Details</h2>
18
19      <div class="form-grid">
20        <div class="form-group">
21          <label>Gender</label>
22          <select name="gender" required>
23            <option value="1">Male</option>
24            <option value="0">Female</option>
25          </select>
26        </div>
27
28        <div class="form-group">
29          <label>Married</label>
30          <select name="married" required>
31            <option value="1">Yes</option>
32            <option value="0">No</option>
33          </select>
34        </div>
35
36        <div class="form-group">
37          <label>Dependents</label>

```



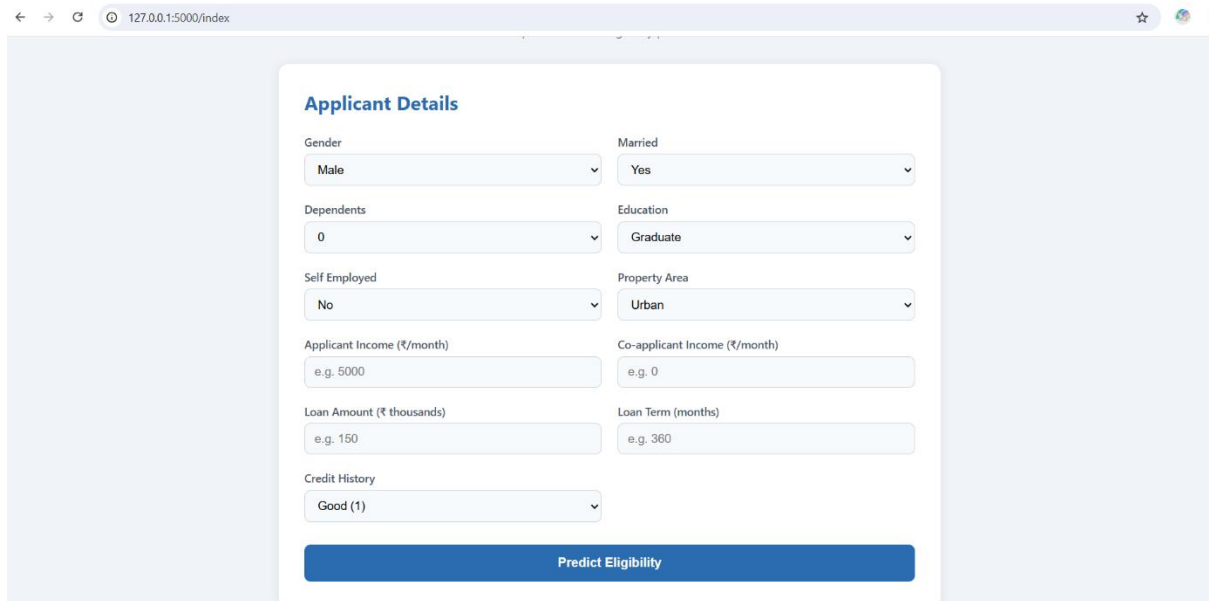
```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>Prediction Result - Smart Lender</title>
6   <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
7 </head>
8 <body>
9   <div class="container">
10     <header>
11       <h1> Smart Lender</h1>
12     </header>
13
14     <div class="result-card {% if 'Approved' in prediction %}approved{% else %}rejected{% endif %}">
15       <h2>Prediction Result</h2>
16       <div class="result-badge">{{ prediction }}</div>
17       <p class="confidence">Model confidence: <strong>{{ confidence }}</strong></p>
18       <a href="/" class="btn-back">+ Check Another Applicant</a>
19     </div>
20   </body>
21 </html>

```

The completed HTML pages provide an interactive interface for the SmartLender application. The **Home Page** welcomes users and provides access to the loan prediction system, while the **Application Form** collects the required applicant information. After submission, the **Result Page** displays whether the loan is **Approved** or **Rejected** based on the prediction generated by the trained machine learning model.

The integration of HTML pages with the Flask application creates a seamless workflow from data entry to prediction, ensuring a smooth and user-friendly experience. The responsive design also allows the application to function effectively across different screen sizes and devices.



The screenshot shows a web browser window with the URL 127.0.0.1:5000/index. The main content is a form titled "Applicant Details". The form contains several input fields and dropdown menus:

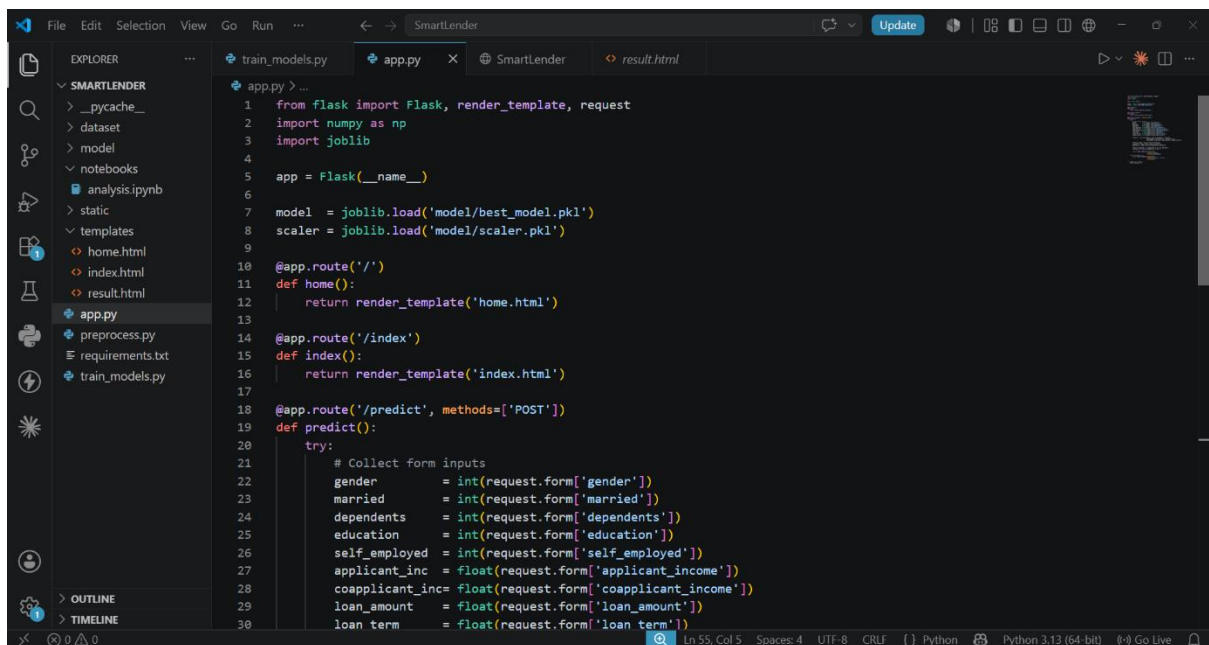
- Gender:** A dropdown menu with "Male" selected.
- Married:** A dropdown menu with "Yes" selected.
- Dependents:** A dropdown menu with "0" selected.
- Education:** A dropdown menu with "Graduate" selected.
- Self Employed:** A dropdown menu with "No" selected.
- Property Area:** A dropdown menu with "Urban" selected.
- Applicant Income (₹/month):** A text input field with "e.g. 5000" as a placeholder.
- Co-applicant Income (₹/month):** A text input field with "e.g. 0" as a placeholder.
- Loan Amount (₹ thousands):** A text input field with "e.g. 150" as a placeholder.
- Loan Term (months):** A text input field with "e.g. 360" as a placeholder.
- Credit History:** A dropdown menu with "Good (1)" selected.

At the bottom of the form is a blue button labeled "Predict Eligibility".

Activity 5.2: Building the Python Script

The Python script forms the core of the SmartLender Loan Prediction System. It was developed to perform data preprocessing, train multiple machine learning models, evaluate their performance, and generate accurate loan approval predictions. The script uses Python libraries such as Pandas, NumPy, Scikit-learn, Joblib, and Flask to implement the complete machine learning workflow.

The script includes functions for loading the dataset, preprocessing the data, scaling features, splitting the dataset into training and testing sets, training different machine learning algorithms, comparing their performance, and saving the best-performing model for deployment in the web application.

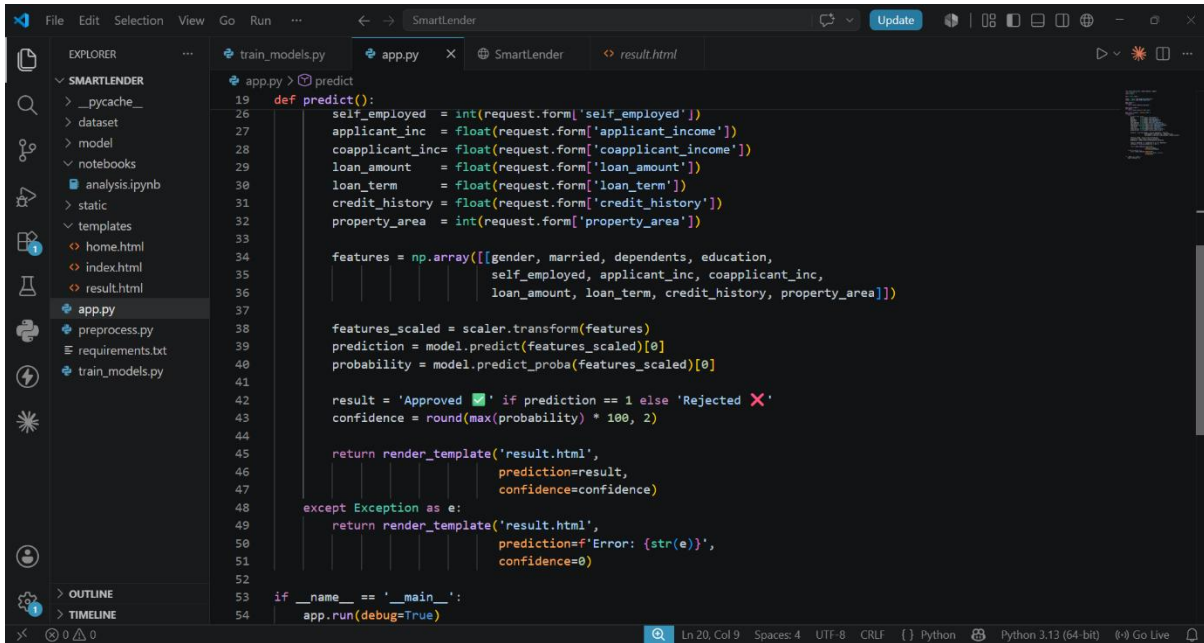


The screenshot shows a code editor with the following Python code in `app.py`:

```

1 from flask import Flask, render_template, request
2 import numpy as np
3 import joblib
4
5 app = Flask(__name__)
6
7 model = joblib.load('model/best_model.pkl')
8 scaler = joblib.load('model/scaler.pkl')
9
10 @app.route('/')
11 def home():
12     return render_template('home.html')
13
14 @app.route('/index')
15 def index():
16     return render_template('index.html')
17
18 @app.route('/predict', methods=['POST'])
19 def predict():
20     try:
21         # Collect form inputs
22         gender = int(request.form['gender'])
23         married = int(request.form['married'])
24         dependents = int(request.form['dependents'])
25         education = int(request.form['education'])
26         self_employed = int(request.form['self_employed'])
27         applicant_inc = float(request.form['applicant_income'])
28         coapplicant_inc = float(request.form['coapplicant_income'])
29         loan_amount = float(request.form['loan_amount'])
30         loan_term = float(request.form['loan_term'])

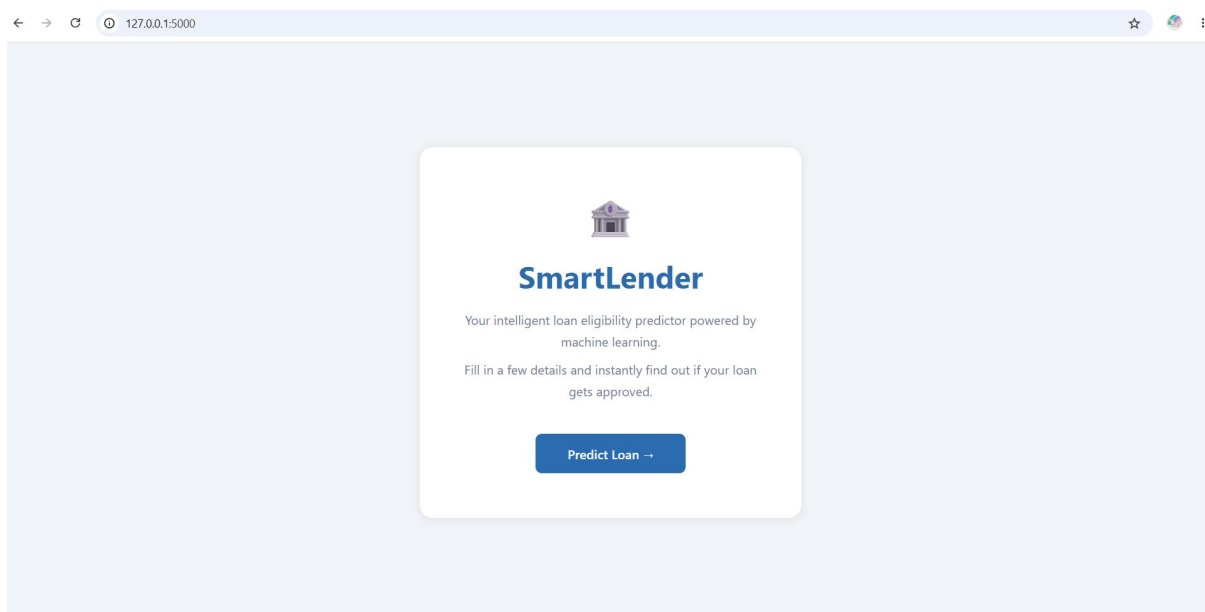
```



```

19 def predict():
20     self_employed = int(request.form['self_employed'])
21     applicant_inc = float(request.form['applicant_income'])
22     coapplicant_inc = float(request.form['coapplicant_income'])
23     loan_amount = float(request.form['loan_amount'])
24     loan_term = float(request.form['loan_term'])
25     credit_history = float(request.form['credit_history'])
26     property_area = int(request.form['property_area'])
27
28     features = np.array([[gender, married, dependents, education,
29                           self_employed, applicant_inc, coapplicant_inc,
30                           loan_amount, loan_term, credit_history, property_area]])
31
32     features_scaled = scaler.transform(features)
33     prediction = model.predict(features_scaled)[0]
34     probability = model.predict_proba(features_scaled)[0]
35
36     result = 'Approved' if prediction == 1 else 'Rejected'
37     confidence = round(max(probability) * 100, 2)
38
39     return render_template('result.html',
40                           prediction=result,
41                           confidence=confidence)
42
43 except Exception as e:
44     return render_template('result.html',
45                           prediction=f'Error: {str(e)}',
46                           confidence=0)
47
48 if __name__ == '__main__':
49     app.run(debug=True)
  
```

The interface after when we run the python file will look like the below image

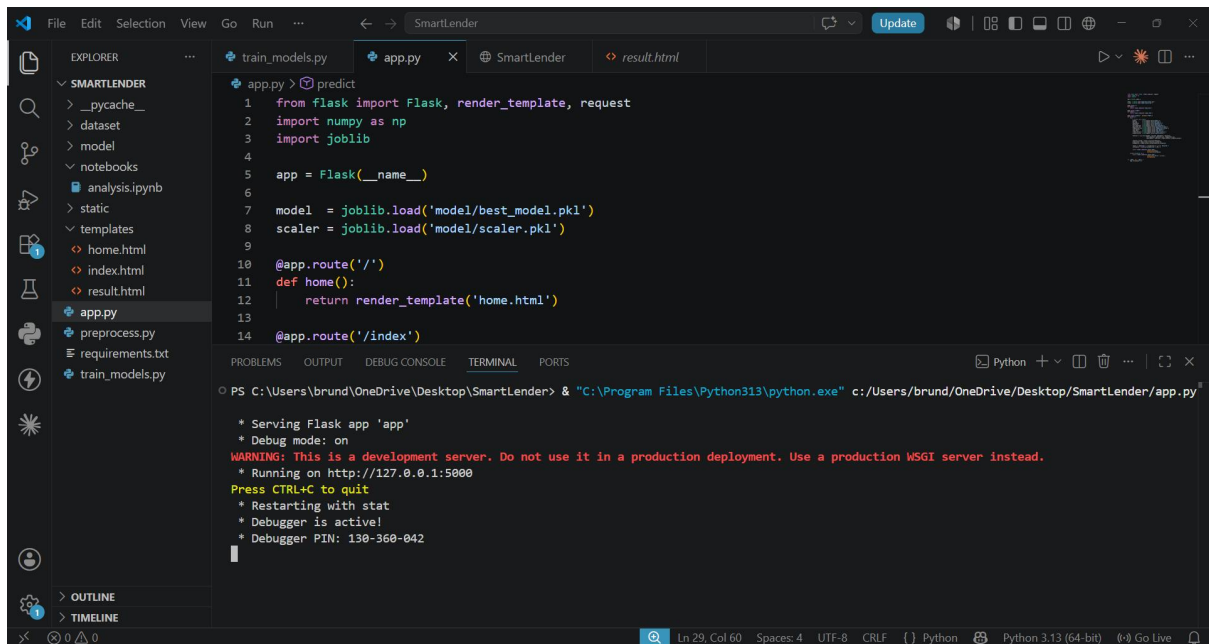


The completed Python script successfully integrates all stages of the machine learning pipeline into a single application. It automates data processing, model training, performance evaluation, and prediction generation, ensuring an efficient and reliable workflow. The best-performing model is saved using **Joblib** and later loaded by the Flask application to provide real-time loan approval predictions. This script serves as the backbone of the SmartLender system and enables accurate, fast, and consistent loan prediction.

Activity 5.3: Run the Application:

After developing the SmartLender Loan Prediction System, the Flask application was executed to integrate the trained machine learning model with the web interface. The application was started using the app.py file, which launched a local web server. Once the server was running successfully, the application could be accessed through a web browser

using the local host URL. This allowed users to interact with the system by entering loan application details and receiving instant loan approval predictions.



The screenshot shows the VS Code editor with the following files in the Explorer: `train_models.py`, `app.py`, `SmartLender`, and `result.html`. The `app.py` file is open, showing the following code:

```

1 from flask import Flask, render_template, request
2 import numpy as np
3 import joblib
4
5 app = Flask(__name__)
6
7 model = joblib.load('model/best_model.pkl')
8 scaler = joblib.load('model/scaler.pkl')
9
10 @app.route('/')
11 def home():
12     return render_template('home.html')
13
14 @app.route('/index')

```

The terminal output shows the following messages:

```

PS C:\Users\brund\OneDrive\Desktop\SmartLender> & "C:\Program Files\Python313\python.exe" c:/Users/brund/OneDrive/Desktop/SmartLender/app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 130-360-042

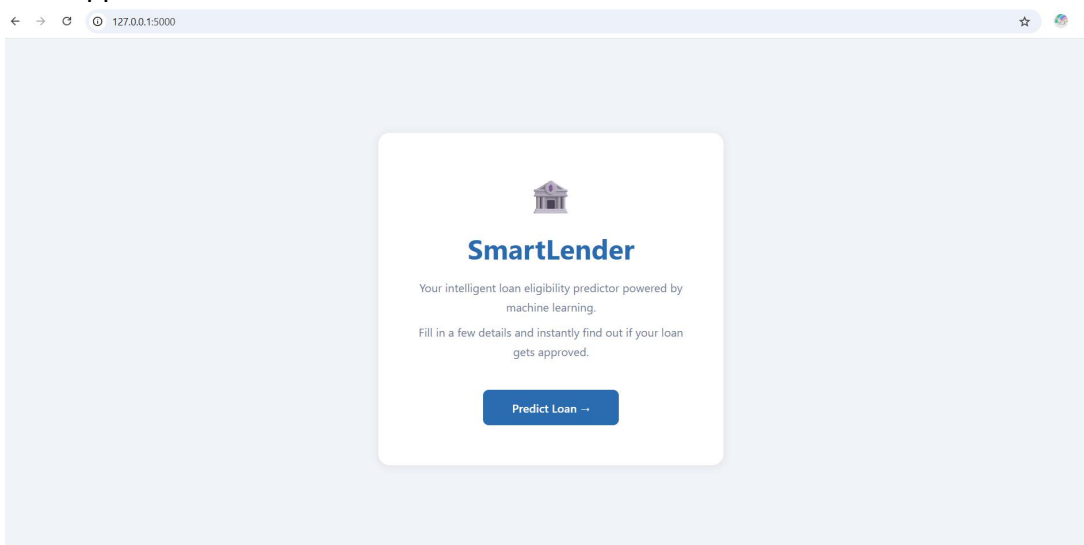
```

Once the application was running, the **Home Page** was displayed in the browser. Users could navigate to the loan application form, enter the required details, and submit the form for prediction. The Flask application processed the input data, loaded the trained machine learning model, and displayed the prediction result as either **Approved** or **Rejected**. This step confirmed the successful deployment and end-to-end functionality of the SmartLender Loan Prediction System.

Exploring the Website's Web Pages:

Home / Generator Page:

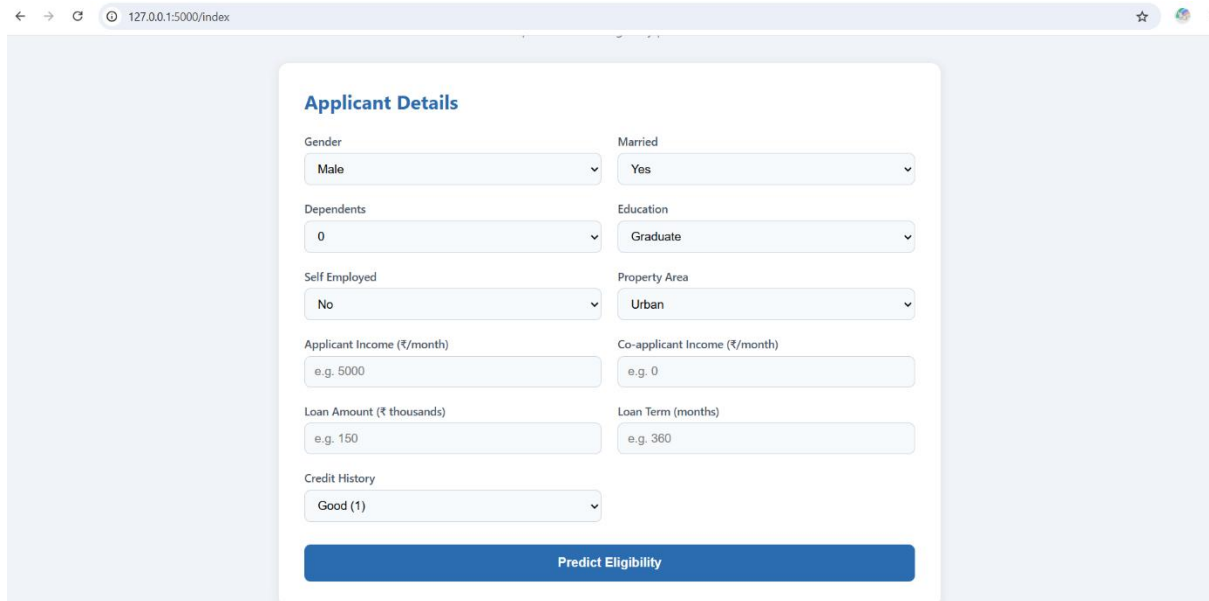
The Home Page is the landing page of the SmartLender Loan Prediction System. It provides a brief introduction to the application and welcomes users with a clean and professional interface. The page includes a "Get Started" or "Predict Loan" button that directs users to the loan application form.



The Home Page serves as the entry point to the application and offers easy navigation to the loan prediction process. Its simple and user-friendly design enhances the overall user experience and provides quick access to the loan application form.

Input Section:

The **Loan Application Page** allows users to enter all the required details needed for loan prediction. The form collects information such as gender, marital status, education, applicant income, co-applicant income, loan amount, loan term, credit history, and property area. These details are validated and submitted to the Flask backend for prediction.



The screenshot shows a web browser window with the URL 127.0.0.1:5000/index. The form is titled "Applicant Details" and contains the following fields:

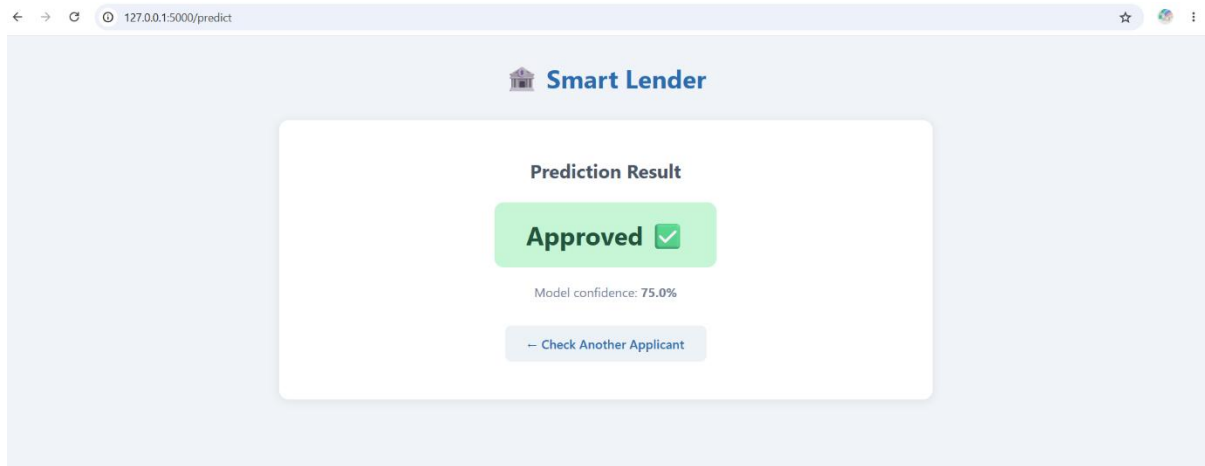
Field	Value
Gender	Male
Married	Yes
Dependents	0
Education	Graduate
Self Employed	No
Property Area	Urban
Applicant Income (₹/month)	e.g. 5000
Co-applicant Income (₹/month)	e.g. 0
Loan Amount (₹ thousands)	e.g. 150
Loan Term (months)	e.g. 360
Credit History	Good (1)

At the bottom of the form is a blue button labeled "Predict Eligibility".

After entering the required information, users can submit the form by clicking the **Predict Loan** button. The application processes the input data, applies the necessary preprocessing steps, and sends it to the trained Random Forest model to generate the loan approval prediction.

Results Section:

The **Result Page** displays the prediction generated by the trained machine learning model after processing the user's input. Based on the applicant's details, the system determines whether the loan application is **Approved** or **Rejected** and presents the result in a clear and easy-to-understand format.



The Result Page provides the final loan prediction along with a user-friendly message indicating the application status. This page confirms that the SmartLender Loan Prediction System has successfully processed the input data and generated an accurate prediction, completing the end-to-end workflow of the application.

Conclusion:

The **SmartLender – Loan Prediction System** successfully demonstrates how machine learning can improve the loan approval process by providing fast, accurate, and reliable predictions. The system analyzes applicant details such as income, credit history, education, and loan amount to determine whether a loan should be approved or rejected. Throughout the project, data preprocessing, feature scaling, model training, and performance evaluation were carried out to develop an effective prediction model.

Among the evaluated algorithms, the **Random Forest Classifier** achieved the best performance and was selected for deployment. The developed Flask web application provides a simple and user-friendly interface for real-time loan prediction. Overall, the project shows the practical application of machine learning in the banking sector by reducing manual effort, improving decision-making, and enabling efficient loan approval.